

Data Engineering

Alka Bhushan

Data Engineering

Definition: Data engineering is the development, operation, and maintenance of data infrastructure, either on-premises or in the cloud (or hybrid or multi-cloud), comprising databases and pipelines to extract, transform, and load data.

Data Characteristics:

- **Volume** : thousand vs million records transfer
- **Velocity** : tracking activity of million vs thousand users
- **Variety** : different data formats at different locations

Depending upon volume, velocity and variety of the data, tools vary for data engineering

Data Engineering Architecture

Applications -> Relational Database -> Batch + Streaming Ingestion -> Data Lake -> ETL -> Data Warehouse -> BI Dashboards/ ML

- Applications:
 - Login events, clicks, sensor readings, transactions, purchases
- Relational Database: stores operational data
 - Handles insert, update, delete, small fast queries
- Batch / streaming pipeline:
 - Streaming : real-time continuous processing
 - Batch: process accumulated data periodically
- Data Lake : store huge amount of raw, semi-processed, structured or unstructured data
- ETL
 - Extract from: logs, APIs, database etc
 - Transform: clean and process data
 - Remove duplicates, standardize format, aggregate, validate)
 - Load: store processed data
- Data Warehouse :
 - convert data from data lake into clean, structured, aggregated and optimized data for fast analytical queries
- BI dashboards / ML : final consumption layer

Data Engineering Tools

Programming Languages : SQL, Python, Java, Scala

Databases:

- Relational databases: MySQL, PostgreSQL,
- Data warehouses: Amazon RedShift, Google BigQuery, Snowflake
- Data lakes: Amazon S3, Hadoop HDFS
- Search and Log Analytics Engine : ElasticSearch

Data Processing and Streaming Systems:

- Batch + near real time stream processing frameworks : Apache Spark (processing engine), Apache Beam (programming model/pipeline abstraction)
 - Stream processing in micro batches
- Real-time Stream processing engines: Apache Flink (advanced), Apache Storm (basic)
- Distributed Event Streaming/Messaging System: Kafka

Data Pipelines (Workflow Orchestration): Apache AirFlow, Apache Nifi

Data Quality and Governance

Governance in MLOps is the set of controls, policies, and processes applied to machine learning operations to ensure:

- proper data usage and privacy,
- compliance with regulations,
- model reliability and fairness, and
- use of well-defined processes throughout the ML lifecycle.

Why Do We Need?

- ML processes are complex and distributed
- Models are often black box
- ML systems depend heavily on data quality and usage
- Regulations like General Data Protection Regulation (GDPR) require strict control over personal data

Two Major Areas of Governance

- Data Governance
- Process Governance

Two Major Areas of Governance

Data Governance

Ensures appropriate use and management of data.

Focus areas:

- Data provenance (where data came from)
- Terms of data collection and usage
- Data quality and freshness
- Detection/removal of PII and sensitive data
- Bias and fairness monitoring
- Data lineage tracking

Process Governance

Uses defined workflows and controls throughout the ML lifecycle.

Focus areas:

- Validation and approval workflows
- Documentation and audit trails
- Compliance checks before deployment
- Review/sign-off mechanisms
- Risk management and accountability

Data Governance

Key Challenges

- **Data Lineage:**
 - Where data originated
 - How datasets were transformed
 - Which features were derived from which sources
 - Downstream impact of data changes
- **Privacy & Compliance Risks**
 - Even anonymized data can sometimes be re-identified.
 - Important regulations:
 - General Data Protection Regulation (GDPR)
 - Data protection and privacy laws
- **Bias in ML Models**
 - Historical data may unintentionally introduce discrimination.
 - Example:
 - Amazon reportedly discontinued an AI recruiting tool after discovering bias against women because historical hiring patterns favored men.

Common Tools: [Apache Atlas](#)

Process Governance

Formalizing ML lifecycle activities with controls, reviews, and accountability.

Goals of Process Governance

- **Ensure Correct Actions at Correct Stages**
 - Model validation before deployment
 - Security review before production release
 - Bias and fairness assessment
 - Approval gates for high-risk models
- **Enable Organizational Oversight**
 - Stakeholders include: Auditors, Risk managers, Compliance teams, Business leadership

Challenges

- ML workflows differ across teams
- End-to-end processes are hard to standardize
- Governance may become too heavyweight
- Teams may bypass overly complex controls

Best Practices

- Automate governance checks in CI/CD pipelines
- Maintain model cards and documentation
- Implement approval workflows
- Use continuous monitoring for drift and bias
- Align governance with Responsible AI principles

Tools with Process Governance Capabilities: MLFlow, Kubeflow, Weights & Biases, Azure Machine Learning, Google Vertex AI,

Apache Spark

Apache Spark Demo

Apache Airflow

Apache Airflow

- A framework for building data engineering pipelines in python
- Created by Airbnb, now an open source Apache Software Foundation Project
- Purpose is to automate and orchestrate complex data pipeline and other workflows
- Allows programmatic authoring, scheduling, and monitoring of workflows as DAGs
- Contains a web server, a scheduler, a metastore, a queueing system, and executors
- Can run as a single instance or can run on a cluster with many executor nodes
- Uses DAGs where each node is a task

When to use ?

- When workflow has clear start and end and run on a schedule
- Prefers python coding to build workflow
 - Provides version control, team collaboration, testing and extendibility
- When pipeline is complex and recurring
- Want to process historical data, rerun failed tasks
- Designed for batches workflows

Airflow: Architecture

- **Web Server (UI):** Provides a visual interface to manage and monitor workflows.
- **Scheduler:** Monitors DAGs and triggers tasks based on time or events.
- **Executor:** Manages how tasks are run, distributing them to workers (e.g., SequentialExecutor, CeleryExecutor, KubernetesExecutor).
- **Workers:** Execute the tasks.
- **Metadata Database:** Stores DAGs, task states, and other metadata.

Basic Concepts

<https://theaisummer.com/apache-airflow-tutorial/#key-components>

Apache Airflow: DAGs key attributes

- Schedule : when workflow should run
- Tasks : discrete units of work that are run on workers
- Task dependencies : the order and conditions under which tasks execute
- Callbacks: actions to take when entire workflows completes
- Additional Parameters: many other operational details

Use Cases

- **ETL (Extract, Transform, Load) pipelines:** Manage and automate data ingestion, transformation, and loading.
- **Machine learning workflows:** Orchestrate training, evaluation, and deployment of machine learning models.
- **Data warehousing:** Build and manage data pipelines for data lakes and warehouses.
- **Ad-hoc job scheduling:** Replace cron with more robust and flexible scheduling.
- Promotes code reusability and collaboration.
- Scalable and fault-tolerant.
- Provides visibility and control over data pipelines.

Apache Airflow Demo

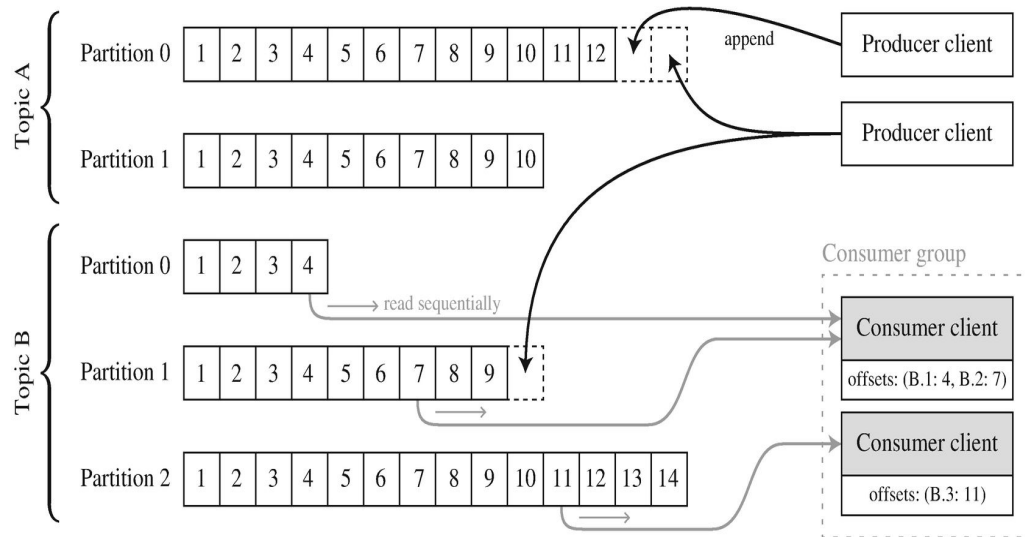
Apache Kafka

Apache Kafka

- An event streaming platform
- Developed by LinkedIn, now an Apache Project
- kafka.apache.org
- Can run on a single node or on a cluster
- Event Streaming:
 - Capture data in real time from event sources like databases, sensors and cloud services etc.
 - Stores for later retrieval
 - Manipulates, processes and reacts in real time as well as retrospectively
 - Routes to different destination technologies

Main Concepts

- **Event:** a record or a message
 - read/write on kafka is an event
 - Contains key, value, timestamps and optional metadata headers
- **Producers:** client applications that publish (write) events to kafka
- **Consumers:** that subscribe to (read and process) these events
- **Topics:** events are organized and stored in topics
 - Topics are folders and files are events
- **Partitions :** each topic has multiple partitions
- **Brokers:** kafka servers to save messages



Apache Kafka Demo

Apache Spark

Apache Airflow

Conda activate mlops

```
export AIRFLOW__CORE__LOAD_EXAMPLES=False
```

airflow standalone